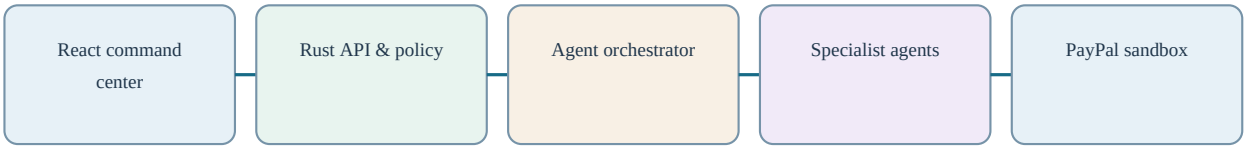
MULTI-AGENT AI PAYMENT SECURITY PLATFORM

40-Phase Technical Development Masterbook

Platform at a glance



Illustrative architecture view — implementation contracts are defined in the relevant phase.

A practical build plan for a three-person engineering team

Team Lead: Sanjay (provisional)

Member 1: _____   Member 2: _____

Includes system architecture, module ownership, 40 phase-by-phase implementation briefs, copy-ready AI coding prompts, testing criteria, integration checkpoints and release guidance.

How to use this masterbook

This is a project implementation plan based on the 40-phase platform previously outlined in this conversation. It is not an official PayPal specification, a guarantee of API availability, or a claim that the system is unhackable. Verify current PayPal API documentation, product eligibility, SDK versions, event types and sponsor requirements before implementation.

Each phase is deliberately organized into five working pages: (1) objective and scope, (2) implementation specification, (3) copy-ready AI coding prompt, (4) testing and security acceptance, and (5) ownership and integration handoff. The pages are intended to be used as working briefs; the team should fill in actual decisions, test evidence and links as work progresses.

Team roles

Team Lead – Sanjay (provisional): owns product direction, architecture approval, cross-team coordination, scope decisions, integration gates and final release. The lead may also own a technical module, but should not become the only person who understands the system.

Member 1 – AI & Agent Intelligence Engineer: owns agent orchestration, specialist agents, risk aggregation, recommendation logic, feedback evaluation and model lifecycle.

Member 2 – Rust Backend & Payment Security Engineer: owns Rust API, database, authentication, policy enforcement, payment adapter, PayPal integration, webhooks, audit and backend security.

Member 3 – Frontend & Platform Integration Engineer: owns React UI, command center, agent workspace, investigation screens, frontend API integration and demo experience. The two member names are intentionally left blank for the team to fill in.

Important engineering rules

• AI output is advisory. Deterministic Rust policy code and authorized payment workflows control financial execution.

• Use PayPal Sandbox for development and demonstrations. Do not use real customer payment data in tests.

• Do not store card numbers, CVVs, passwords, API secrets or unnecessary sensitive identity data.

• Use decimal-safe money representations and explicit currency handling.

• Verify webhook authenticity, handle duplicates and reconcile against authoritative payment state.

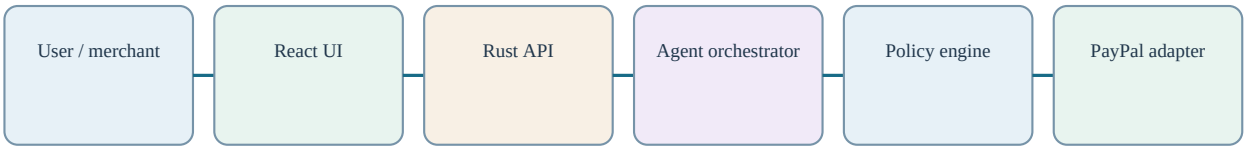
• Model learning must use verified labels, versioned datasets, evaluation gates and rollback.

• Do not expose model chain-of-thought. Show evidence, reason codes, confidence, model version and policy result.

• Build a modular monolith first. Split services only when a measured need justifies it.

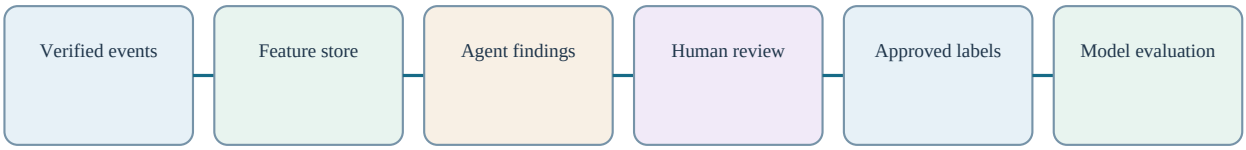
Platform architecture overview

Core request and payment decision path



Illustrative architecture view — implementation contracts are defined in the relevant phase.

Data and learning path



Illustrative architecture view — implementation contracts are defined in the relevant phase.

Trust boundaries

- Browser: untrusted client; it can request actions but cannot authorize payment execution.
- Rust API: authenticates users, validates input, enforces tenant boundaries and coordinates workflows.
- Agent runtime: receives scoped data and tools; outputs are untrusted until schema and policy validation.
- Policy engine: deterministic, versioned and testable; decides whether an action may proceed.
- PayPal adapter: server-side only; uses securely stored credentials and documented API flows.
- Learning pipeline: separated from live decisions; models are promoted only after evaluation and approval.

Agent catalogue and responsibility boundaries

ID	Agent	Primary responsibility	Must not do
00	Master Orchestrator	Plan and coordinate agent tasks	Execute payments or bypass policy
01	Transaction Analyst	Analyze transaction patterns	Declare final fraud verdict
02	Identity Intelligence	Evaluate approved identity signals	Invent or verify identity independently
03	Merchant Intelligence	Analyze merchant risk indicators	Make unsupported reputation claims
04	Device Intelligence	Analyze permitted device/session signals	Perform covert tracking
05	Behavior Monitor	Analyze event sequences	Infer sensitive traits
06	Payment Intelligence	Analyze payment outcomes	Fabricate payment state
07	Checkout Optimizer	Recommend checkout improvements	Use deceptive dark patterns
08	Customer Intelligence	Generate purpose-limited customer insights	Infer protected attributes
09	Recommendation Engine	Rank relevant products/actions	Hide sponsored influence
10	Refund Intelligence	Prepare refund eligibility assessment	Execute unauthorized refunds
11	Dispute Investigator	Assemble source-backed evidence	Fabricate evidence
12	Operations Intelligence	Identify operational bottlenecks	Take unapproved external actions

All agents share a common output envelope: agent ID, version, task ID, evidence references, structured findings, confidence/uncertainty, limitations, timestamp and trace ID. The orchestrator validates this envelope before aggregation.

Three-person ownership and synchronization

Workstream	Primary owner	Review / partner	Main output
Product, architecture, release	Sanjay – Team Lead (provisional)	All members	PRD, decisions, checkpoints, release
Agent runtime and intelligence	Member 1 – AI Lead	Team Lead + Backend Lead	Agent modules, model evaluation, schemas
Rust API, security, payments	Member 2 – Backend Lead	Team Lead + AI Lead	Rust services, policy, PayPal adapter
React UI and workflows	Member 3 – Frontend Lead	Team Lead + AI Lead	Dashboards, case flows, API integration
Cross-system testing	All members	Team Lead coordinates	Contract, integration, security evidence

Checkpoint rhythm

• Daily: 10–15 minute sync; each person reports completed work, next work and blockers.

• At each checkpoint: freeze API/event schemas, merge behind feature flags, run contract tests and record evidence.

• Pull requests: one author, one technical reviewer and one integration reviewer for security/payment-sensitive changes.

• Definition of done: code, tests, docs, logs/metrics, security review and a working demo path.

• Never merge directly to the release branch. Keep integration branch green and tag known-good checkpoints.

PHASE 01 — Product Requirements & Problem Definition

Primary owner: Product Lead / Team Lead

Purpose

Define the real payment, fraud, commerce and operational problems the platform will solve. Convert broad goals into measurable user journeys and an achievable hackathon MVP.

Scope and work packages

• Interview or model the needs of merchants, payment operations teams, risk analysts and customers.

• Define supported use cases: transaction risk assessment, checkout friction, merchant risk, refunds, disputes, recommendations and operations intelligence.

• Separate MVP requirements from later-stage capabilities; explicitly exclude unsupported claims such as guaranteed fraud prevention.

• Define measurable outcomes: decision latency, precision/recall, false-positive rate, payment completion rate, review workload and audit completeness.

• Document user roles, data permissions, legal/privacy assumptions, non-functional requirements and demo scenarios.

Expected deliverables

A signed-off PRD, user stories, use-case map, measurable success criteria, MVP boundary and initial risk register.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 01 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 01 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 01: Product Requirements & Problem Definition in the Multi-Agent AI Payment Security Platform. Primary owner: Product Lead / Team Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Define the real payment, fraud, commerce and operational problems the platform will solve. Convert broad goals into measurable user journeys and an achievable hackathon MVP. Implement the following scope: Interview or model the needs of merchants, payment operations teams, risk analysts and customers. Define supported use cases: transaction risk assessment, checkout friction, merchant risk, refunds, disputes, recommendations and operations intelligence. Separate MVP requirements from later-stage capabilities; explicitly exclude unsupported claims such as guaranteed fraud prevention. Define measurable outcomes: decision latency, precision/recall, false-positive rate, payment completion rate, review workload and audit completeness. Document user roles, data permissions, legal/privacy assumptions, non-functional requirements and demo scenarios. Deliverables: A signed-off PRD, user stories, use-case map, measurable success criteria, MVP boundary and initial risk register. Implementation directive: Build the requirements pack and validate that each proposed feature has a user, an input, an output, a measurable value and a safe failure mode. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 01 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 01 — Ownership, integration & handoff

Ownership

Primary owner: Product Lead / Team Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 02 — System Architecture & Technical Design

Primary owner: Team Lead + all members

Purpose

Create the end-to-end architecture and agree on module boundaries before parallel coding begins.

Scope and work packages

• Choose a modular Rust backend first; keep independently deployable services as a later option, not a default.

• Define bounded contexts: identity/access, agent runtime, risk, policy, payments, PayPal adapter, learning, analytics and UI.

• Define synchronous APIs and asynchronous event contracts; specify ownership of every database table and event.

• Document trust boundaries, secrets flow, tenant isolation, failure domains and human approval points.

• Create sequence diagrams for a payment assessment, a payment state update, a dispute investigation and a model feedback cycle.

Expected deliverables

Architecture decision records, component diagram, deployment diagram, sequence diagrams, data-flow map and interface ownership matrix.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 02 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 02 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 02: System Architecture & Technical Design in the Multi-Agent AI Payment Security Platform. Primary owner: Team Lead + all members. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Create the end-to-end architecture and agree on module boundaries before parallel coding begins. Implement the following scope: Choose a modular Rust backend first; keep independently deployable services as a later option, not a default. Define bounded contexts: identity/access, agent runtime, risk, policy, payments, PayPal adapter, learning, analytics and UI. Define synchronous APIs and asynchronous event contracts; specify ownership of every database table and event. Document trust boundaries, secrets flow, tenant isolation, failure domains and human approval points. Create sequence diagrams for a payment assessment, a payment state update, a dispute investigation and a model feedback cycle. Deliverables: Architecture decision records, component diagram, deployment diagram, sequence diagrams, data-flow map and interface ownership matrix. Implementation directive: Produce a reviewable architecture with explicit trust boundaries, API contracts and failure handling. Do not begin implementation until all three members approve the contracts. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 02 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 02 — Ownership, integration & handoff

Ownership

Primary owner: Team Lead + all members. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 03 — Repository, Workspace & Developer Experience

Primary owner: Backend Lead

Purpose

Create a reproducible repository and shared development workflow for a three-person team.

Scope and work packages

• Create Rust workspace crates and a separate React TypeScript frontend application.

• Add formatting, linting, test commands, environment templates and local Docker Compose services.

• Set branch protection, pull-request templates, issue templates, CODEOWNERS and semantic commit guidance.

• Add CI checks for Rust formatting, Clippy, unit tests, frontend lint/typecheck/build and dependency auditing.

• Provide one documented command sequence to start the database, backend and frontend locally.

Expected deliverables

Repository skeleton, CI workflow, developer setup guide, environment example and contribution rules.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 03 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 03 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 03: Repository, Workspace & Developer Experience in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Create a reproducible repository and shared development workflow for a three-person team. Implement the following scope: Create Rust workspace crates and a separate React TypeScript frontend application. Add formatting, linting, test commands, environment templates and local Docker Compose services. Set branch protection, pull-request templates, issue templates, CODEOWNERS and semantic commit guidance. Add CI checks for Rust formatting, Clippy, unit tests, frontend lint/typecheck/build and dependency auditing. Provide one documented command sequence to start the database, backend and frontend locally. Deliverables: Repository skeleton, CI workflow, developer setup guide, environment example and contribution rules. Implementation directive: Build a clean, reproducible workspace. Never commit credentials, production data, generated secrets or real payment information. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 03 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 03 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 04 — Database Architecture & Data Governance

Primary owner: Backend Lead

Purpose

Design durable, auditable storage for transactions, risk assessments, agents, decisions and feedback.

Scope and work packages

• Model organizations, users, roles, agent identities, transactions, payment attempts, orders, assessments, evidence, cases, policies and audit events.

• Use stable identifiers, timestamps, tenant keys, status enums and explicit relationships.

• Define retention, deletion, minimization and access rules for personal and financial data.

• Add migrations, constraints, indexes, transaction boundaries and query performance baselines.

• Separate raw event storage, normalized business records, derived features and model-training datasets.

Expected deliverables

ER diagram, migration set, data dictionary, retention plan, index plan and database access rules.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 04 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 04 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 04: Database Architecture & Data Governance in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Design durable, auditable storage for transactions, risk assessments, agents, decisions and feedback. Implement the following scope: Model organizations, users, roles, agent identities, transactions, payment attempts, orders, assessments, evidence, cases, policies and audit events. Use stable identifiers, timestamps, tenant keys, status enums and explicit relationships. Define retention, deletion, minimization and access rules for personal and financial data. Add migrations, constraints, indexes, transaction boundaries and query performance baselines. Separate raw event storage, normalized business records, derived features and model-training datasets. Deliverables: ER diagram, migration set, data dictionary, retention plan, index plan and database access rules. Implementation directive: Implement a normalized and tenant-aware schema with migrations and tests. Avoid storing card data or unnecessary sensitive personal data. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 04 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 04 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 05 — Authentication, Authorization & Tenant Isolation

Primary owner: Backend Lead

Purpose

Secure users, organizations, sessions, APIs and role-scoped access.

Scope and work packages

• Implement authentication using a vetted identity approach; do not invent password cryptography.

• Define roles such as owner, risk analyst, finance operator, reviewer and read-only auditor.

• Enforce organization and resource-level authorization on every protected endpoint.

• Secure sessions, refresh flows, account recovery, revocation and rate limits.

• Add tests proving one tenant cannot access another tenant's records.

Expected deliverables

Authentication module, RBAC policy matrix, tenant isolation tests, session lifecycle and security configuration.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 05 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 05 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 05: Authentication, Authorization & Tenant Isolation in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Secure users, organizations, sessions, APIs and role-scoped access. Implement the following scope: Implement authentication using a vetted identity approach; do not invent password cryptography. Define roles such as owner, risk analyst, finance operator, reviewer and read-only auditor. Enforce organization and resource-level authorization on every protected endpoint. Secure sessions, refresh flows, account recovery, revocation and rate limits. Add tests proving one tenant cannot access another tenant's records. Deliverables: Authentication module, RBAC policy matrix, tenant isolation tests, session lifecycle and security configuration. Implementation directive: Implement secure authentication and authorization using established libraries. Deny by default and verify access on the server for every request. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 05 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 05 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 06 — Rust Backend Core & API Foundation

Primary owner: Backend Lead

Purpose

Build the core HTTP application and shared Rust service conventions.

Scope and work packages

• Set up Axum routes, Tokio runtime, typed request/response DTOs and application state.

• Implement structured errors, request IDs, tracing, configuration validation and health endpoints.

• Use async database access with bounded pools and timeouts.

• Add request size limits, input validation, CORS policy and secure response headers.

• Define versioned API routing and OpenAPI generation or maintained API specifications.

Expected deliverables

Running Rust API, standard error format, health endpoints, configuration validation and API contract.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 06 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 06 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 06: Rust Backend Core & API Foundation in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Build the core HTTP application and shared Rust service conventions. Implement the following scope: Set up Axum routes, Tokio runtime, typed request/response DTOs and application state. Implement structured errors, request IDs, tracing, configuration validation and health endpoints. Use async database access with bounded pools and timeouts. Add request size limits, input validation, CORS policy and secure response headers. Define versioned API routing and OpenAPI generation or maintained API specifications. Deliverables: Running Rust API, standard error format, health endpoints, configuration validation and API contract. Implementation directive: Create a maintainable Axum foundation with typed boundaries, observable requests, safe defaults and tests for malformed and oversized requests. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 06 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 06 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 07 — Agent Communication & Event Bus

Primary owner: AI Lead + Backend Lead

Purpose

Provide authenticated, typed and traceable communication between agents and services.

Scope and work packages

• Define versioned message envelopes with sender, recipient, correlation ID, timestamp, expiry, schema version and payload type.

• Use NATS or an equivalent broker for asynchronous events; keep request-response APIs for bounded synchronous operations.

• Add authentication, authorization, payload validation, replay protection and idempotent consumers.

• Implement retry with backoff, dead-letter handling, ordering assumptions and poison-message controls.

• Propagate trace context so an investigation can be followed across all participating agents.

Expected deliverables

Event catalogue, message schemas, broker configuration, retry policy and integration tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 07 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 07 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 07: Agent Communication & Event Bus in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Provide authenticated, typed and traceable communication between agents and services. Implement the following scope: Define versioned message envelopes with sender, recipient, correlation ID, timestamp, expiry, schema version and payload type. Use NATS or an equivalent broker for asynchronous events; keep request-response APIs for bounded synchronous operations. Add authentication, authorization, payload validation, replay protection and idempotent consumers. Implement retry with backoff, dead-letter handling, ordering assumptions and poison-message controls. Propagate trace context so an investigation can be followed across all participating agents. Deliverables: Event catalogue, message schemas, broker configuration, retry policy and integration tests. Implementation directive: Implement secure agent messaging. Treat every message as untrusted input and reject invalid, expired, duplicated or unauthorized messages. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 07 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 07 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 08 — Agent Identity & Capability Registry

Primary owner: AI Lead + Backend Lead

Purpose

Give each agent a verifiable identity and narrowly scoped capabilities.

Scope and work packages

• Register agents with immutable IDs, version, owner, purpose, model/tool configuration and lifecycle state.

• Define capabilities such as read-risk-signals, create-assessment, draft-response and request-review.

• Issue short-lived credentials or signed workload identities; rotate and revoke credentials.

• Enforce tool permissions at the tool gateway, not only inside prompts.

• Maintain a registry of approved models, tools, versions and allowed data classes.

Expected deliverables

Agent registry, capability catalogue, lifecycle API, credential rotation and permission tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 08 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 08 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 08: Agent Identity & Capability Registry in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Give each agent a verifiable identity and narrowly scoped capabilities. Implement the following scope: Register agents with immutable IDs, version, owner, purpose, model/tool configuration and lifecycle state. Define capabilities such as read-risk-signals, create-assessment, draft-response and request-review. Issue short-lived credentials or signed workload identities; rotate and revoke credentials. Enforce tool permissions at the tool gateway, not only inside prompts. Maintain a registry of approved models, tools, versions and allowed data classes. Deliverables: Agent registry, capability catalogue, lifecycle API, credential rotation and permission tests. Implementation directive: Implement least-privilege agent identity. An agent must not gain permissions by requesting them in natural language. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 08 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 08 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 09 — Deterministic Policy & Decision Engine

Primary owner: Backend Lead

Purpose

Keep financial and security policy enforcement outside probabilistic model reasoning.

Scope and work packages

• Create versioned rules for risk thresholds, review requirements, spending limits and prohibited actions.

• Represent policy inputs and outputs with typed schemas and deterministic evaluation.

• Support allow, deny, step-up verification and manual review outcomes.

• Require policy version, evidence references and reason codes in each decision.

• Add simulation mode, policy test fixtures, approval for policy changes and rollback.

Expected deliverables

Policy engine crate, policy schema, versioning, decision explanations and exhaustive tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 09 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 09 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 09: Deterministic Policy & Decision Engine in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Keep financial and security policy enforcement outside probabilistic model reasoning. Implement the following scope: Create versioned rules for risk thresholds, review requirements, spending limits and prohibited actions. Represent policy inputs and outputs with typed schemas and deterministic evaluation. Support allow, deny, step-up verification and manual review outcomes. Require policy version, evidence references and reason codes in each decision. Add simulation mode, policy test fixtures, approval for policy changes and rollback. Deliverables: Policy engine crate, policy schema, versioning, decision explanations and exhaustive tests. Implementation directive: Build a deterministic policy engine. AI may recommend; only validated policy and authorized payment code may permit execution. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 09 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 09 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 10 — Audit, Observability & Incident Telemetry

Primary owner: Backend Lead + UI Lead

Purpose

Make every material system action traceable and operationally observable.

Scope and work packages

• Record actor, agent, request ID, policy version, model version, evidence references and outcome.

• Use structured logs, metrics and distributed traces; redact secrets and sensitive fields.

• Create audit event integrity controls and restricted access to audit records.

• Monitor queue lag, agent failures, payment API latency, webhook errors and model quality.

• Define incident severity, alert routing and operational runbooks.

Expected deliverables

Audit event schema, dashboards, alert rules, trace correlation and incident response runbook.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 10 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 10 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 10: Audit, Observability & Incident Telemetry in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead + UI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Make every material system action traceable and operationally observable. Implement the following scope: Record actor, agent, request ID, policy version, model version, evidence references and outcome. Use structured logs, metrics and distributed traces; redact secrets and sensitive fields. Create audit event integrity controls and restricted access to audit records. Monitor queue lag, agent failures, payment API latency, webhook errors and model quality. Define incident severity, alert routing and operational runbooks. Deliverables: Audit event schema, dashboards, alert rules, trace correlation and incident response runbook. Implementation directive: Implement observability without logging secrets or full payment credentials. Demonstrate tracing from a user request to the final policy outcome. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 10 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 10 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead + UI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 11 — Master Agent Orchestrator

Primary owner: AI Lead

Purpose

Coordinate specialist agents without allowing the orchestrator to bypass policy controls.

Scope and work packages

• Define task planning, agent selection, execution graph, deadlines and bounded parallelism.

• Validate agent outputs against typed schemas before accepting them.

• Handle partial results, timeouts, conflicting findings and unavailable agents.

• Preserve provenance: which agent supplied each finding and what evidence it used.

• Return a structured assessment proposal to the deterministic policy engine.

Expected deliverables

Orchestrator crate, task graph, timeout/fallback strategy, traceable agent execution and tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 11 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 11 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 11: Master Agent Orchestrator in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Coordinate specialist agents without allowing the orchestrator to bypass policy controls. Implement the following scope: Define task planning, agent selection, execution graph, deadlines and bounded parallelism. Validate agent outputs against typed schemas before accepting them. Handle partial results, timeouts, conflicting findings and unavailable agents. Preserve provenance: which agent supplied each finding and what evidence it used. Return a structured assessment proposal to the deterministic policy engine. Deliverables: Orchestrator crate, task graph, timeout/fallback strategy, traceable agent execution and tests. Implementation directive: Build an orchestrator that coordinates specialists, validates outputs and never directly executes payment actions. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 11 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 11 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 12 — Transaction Pattern Analyst Agent

Primary owner: AI Lead

Purpose

Analyze transaction-level signals and identify unusual patterns for review.

Scope and work packages

• Consume only authorized, minimized transaction features.

• Calculate frequency, amount deviation, velocity, sequence and historical-baseline signals.

• Distinguish deterministic indicators from model-generated explanations.

• Return a typed score, evidence list, confidence and limitations.

• Evaluate on labeled or synthetic test data and track false positives and false negatives.

Expected deliverables

Agent implementation, feature specification, evaluation dataset description and quality report.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 12 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 12 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 12: Transaction Pattern Analyst Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Analyze transaction-level signals and identify unusual patterns for review. Implement the following scope: Consume only authorized, minimized transaction features. Calculate frequency, amount deviation, velocity, sequence and historical-baseline signals. Distinguish deterministic indicators from model-generated explanations. Return a typed score, evidence list, confidence and limitations. Evaluate on labeled or synthetic test data and track false positives and false negatives. Deliverables: Agent implementation, feature specification, evaluation dataset description and quality report. Implementation directive: Build a transaction analyst that returns evidence-backed signals. Do not treat its score as a final fraud verdict. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 12 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 12 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 13 — Identity Verification Intelligence Agent

Primary owner: AI Lead

Purpose

Assess available identity and authentication evidence without making unsupported identity claims.

Scope and work packages

• Consume approved verification results and account security signals.

• Detect mismatches, recent credential changes and inconsistent verification states.

• Avoid collecting biometrics or sensitive identity documents unless the product has a lawful, secure need.

• Return evidence references, confidence, reason codes and recommended step-up verification.

• Test missing, stale, contradictory and unavailable verification signals.

Expected deliverables

Identity signal contract, agent, privacy controls and step-up recommendation tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 13 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 13 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 13: Identity Verification Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Assess available identity and authentication evidence without making unsupported identity claims. Implement the following scope: Consume approved verification results and account security signals. Detect mismatches, recent credential changes and inconsistent verification states. Avoid collecting biometrics or sensitive identity documents unless the product has a lawful, secure need. Return evidence references, confidence, reason codes and recommended step-up verification. Test missing, stale, contradictory and unavailable verification signals. Deliverables: Identity signal contract, agent, privacy controls and step-up recommendation tests. Implementation directive: Build an evidence-based identity signal analyst. It must not impersonate a regulated identity verification provider. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 13 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 13 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 14 — Merchant Risk Intelligence Agent

Primary owner: AI Lead

Purpose

Assess merchant-level operational and transaction risk using available, authorized data.

Scope and work packages

• Analyze merchant history, dispute/refund patterns, fulfillment indicators and data quality.

• Separate merchant-level trends from individual customer risk.

• Use minimum sample thresholds and confidence intervals where appropriate.

• Detect data drift and explain why a merchant indicator changed.

• Provide reviewer-friendly evidence and avoid unsupported reputation assertions.

Expected deliverables

Merchant feature definitions, agent output schema, calibration report and review UI requirements.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 14 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 14 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 14: Merchant Risk Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Assess merchant-level operational and transaction risk using available, authorized data. Implement the following scope: Analyze merchant history, dispute/refund patterns, fulfillment indicators and data quality. Separate merchant-level trends from individual customer risk. Use minimum sample thresholds and confidence intervals where appropriate. Detect data drift and explain why a merchant indicator changed. Provide reviewer-friendly evidence and avoid unsupported reputation assertions. Deliverables: Merchant feature definitions, agent output schema, calibration report and review UI requirements. Implementation directive: Implement merchant intelligence with evidence provenance, sample-size awareness and explicit uncertainty. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 14 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 14 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 15 — Device & Session Intelligence Agent

Primary owner: AI Lead

Purpose

Identify unusual device/session signals while respecting privacy and platform limitations.

Scope and work packages

• Use permitted device/session signals, authentication events and coarse consistency indicators.

• Detect abrupt changes, impossible sequence assumptions only when supported by data, and repeated suspicious sessions.

• Avoid covert tracking, invasive fingerprinting or collection beyond stated purpose.

• Return explainable signals with retention and access limits.

• Test shared devices, travel, VPNs, browser updates and missing device data to reduce false positives.

Expected deliverables

Device/session feature contract, privacy notes, agent and scenario tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 15 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 15 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 15: Device & Session Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Identify unusual device/session signals while respecting privacy and platform limitations. Implement the following scope: Use permitted device/session signals, authentication events and coarse consistency indicators. Detect abrupt changes, impossible sequence assumptions only when supported by data, and repeated suspicious sessions. Avoid covert tracking, invasive fingerprinting or collection beyond stated purpose. Return explainable signals with retention and access limits. Test shared devices, travel, VPNs, browser updates and missing device data to reduce false positives. Deliverables: Device/session feature contract, privacy notes, agent and scenario tests. Implementation directive: Build privacy-conscious device intelligence. Do not label a device or person fraudulent based on one weak signal. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 15 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 15 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 16 — Behavior Sequence Monitor Agent

Primary owner: AI Lead

Purpose

Analyze event sequences around account and checkout activity.

Scope and work packages

• Define event taxonomy for login, cart, checkout, authentication and payment attempts.

• Detect unusual sequence patterns and rapid repeated attempts.

• Account for legitimate accessibility, network and user behavior variation.

• Return event references, pattern labels and confidence.

• Evaluate on representative synthetic scenarios and measure false alarms.

Expected deliverables

Event taxonomy, sequence detector, test scenarios and explainability output.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 16 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 16 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 16: Behavior Sequence Monitor Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Analyze event sequences around account and checkout activity. Implement the following scope: Define event taxonomy for login, cart, checkout, authentication and payment attempts. Detect unusual sequence patterns and rapid repeated attempts. Account for legitimate accessibility, network and user behavior variation. Return event references, pattern labels and confidence. Evaluate on representative synthetic scenarios and measure false alarms. Deliverables: Event taxonomy, sequence detector, test scenarios and explainability output. Implementation directive: Build a sequence-analysis agent with robust missing-event handling and no hidden surveillance behavior. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 16 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 16 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 17 — Collaborative Risk Aggregation Engine

Primary owner: AI Lead + Backend Lead

Purpose

Combine specialist outputs into a calibrated risk assessment.

Scope and work packages

• Define common normalized signal schema and missing-data behavior.

• Compare agent outputs, identify contradictions and prevent double-counting correlated signals.

• Use a calibrated aggregation method with versioned weights or a trained model.

• Return risk band, reasons, uncertainty and required review action.

• Benchmark precision, recall, false-positive rate, false-negative rate and decision latency.

Expected deliverables

Aggregation engine, calibration report, decision explanation and policy integration contract.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 17 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 17 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 17: Collaborative Risk Aggregation Engine in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Combine specialist outputs into a calibrated risk assessment. Implement the following scope: Define common normalized signal schema and missing-data behavior. Compare agent outputs, identify contradictions and prevent double-counting correlated signals. Use a calibrated aggregation method with versioned weights or a trained model. Return risk band, reasons, uncertainty and required review action. Benchmark precision, recall, false-positive rate, false-negative rate and decision latency. Deliverables: Aggregation engine, calibration report, decision explanation and policy integration contract. Implementation directive: Build a transparent risk aggregator. It must expose uncertainty and route ambiguous/high-impact cases to review. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 17 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 17 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 18 — Payment Intelligence Agent

Primary owner: AI Lead

Purpose

Analyze payment outcomes and operational signals to improve payment reliability.

Scope and work packages

• Classify payment states and failure categories using authoritative gateway responses.

• Analyze retry history, timeouts, declines and duplicate attempts.

• Recommend safe retry or alternate next steps without bypassing issuer/gateway controls.

• Distinguish transient failures from hard declines.

• Measure payment completion and retry outcomes.

Expected deliverables

Payment event taxonomy, agent, retry recommendation schema and evaluation report.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 18 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 18 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 18: Payment Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Analyze payment outcomes and operational signals to improve payment reliability. Implement the following scope: Classify payment states and failure categories using authoritative gateway responses. Analyze retry history, timeouts, declines and duplicate attempts. Recommend safe retry or alternate next steps without bypassing issuer/gateway controls. Distinguish transient failures from hard declines. Measure payment completion and retry outcomes. Deliverables: Payment event taxonomy, agent, retry recommendation schema and evaluation report. Implementation directive: Build payment intelligence that recommends safe next steps; it must never fabricate a successful payment state. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 18 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 18 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 19 — Checkout Optimization Agent

Primary owner: AI Lead + UI Lead

Purpose

Identify checkout friction and recommend improvements without manipulating users.

Scope and work packages

• Analyze funnel events, form errors, latency and payment option availability.

• Recommend fewer unnecessary steps and clearer error recovery.

• Support accessible UI and transparent presentation of fees and payment options.

• Avoid discriminatory personalization or deceptive urgency patterns.

• Measure abandonment and conversion with controlled experiments.

Expected deliverables

Checkout analytics schema, recommendation agent, UI integration and experiment plan.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 19 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 19 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 19: Checkout Optimization Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + UI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Identify checkout friction and recommend improvements without manipulating users. Implement the following scope: Analyze funnel events, form errors, latency and payment option availability. Recommend fewer unnecessary steps and clearer error recovery. Support accessible UI and transparent presentation of fees and payment options. Avoid discriminatory personalization or deceptive urgency patterns. Measure abandonment and conversion with controlled experiments. Deliverables: Checkout analytics schema, recommendation agent, UI integration and experiment plan. Implementation directive: Build a checkout optimization agent focused on clarity, accessibility and measurable friction reduction. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 19 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 19 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + UI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 20 — Customer Intelligence Agent

Primary owner: AI Lead

Purpose

Generate useful customer-level insights from consented and relevant business activity.

Scope and work packages

• Create behavior segments using purpose-limited features.

• Identify churn indicators and engagement patterns.

• Support customer-level explanations and data correction.

• Prevent sensitive-attribute inference and unfair proxy use.

• Evaluate segment stability and business usefulness.

Expected deliverables

Customer feature catalogue, segmentation logic, privacy controls and evaluation report.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 20 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 20 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 20: Customer Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Generate useful customer-level insights from consented and relevant business activity. Implement the following scope: Create behavior segments using purpose-limited features. Identify churn indicators and engagement patterns. Support customer-level explanations and data correction. Prevent sensitive-attribute inference and unfair proxy use. Evaluate segment stability and business usefulness. Deliverables: Customer feature catalogue, segmentation logic, privacy controls and evaluation report. Implementation directive: Build customer intelligence using purpose limitation, explainability and fairness checks. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 20 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 20 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 21 — Recommendation Engine

Primary owner: AI Lead

Purpose

Recommend products, payment options or next actions using relevant context.

Scope and work packages

• Build candidate generation and ranking stages.

• Use availability, budget, user preference and business constraints.

• Explain why each recommendation appears and identify sponsored or commercial ranking influences.

• Support feedback, cold-start behavior and fallback recommendations.

• Measure relevance, diversity, coverage and conversion without over-personalization.

Expected deliverables

Recommendation API, ranking model, explanation schema and offline/online evaluation plan.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 21 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 21 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 21: Recommendation Engine in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Recommend products, payment options or next actions using relevant context. Implement the following scope: Build candidate generation and ranking stages. Use availability, budget, user preference and business constraints. Explain why each recommendation appears and identify sponsored or commercial ranking influences. Support feedback, cold-start behavior and fallback recommendations. Measure relevance, diversity, coverage and conversion without over-personalization. Deliverables: Recommendation API, ranking model, explanation schema and offline/online evaluation plan. Implementation directive: Build a transparent recommender with safe fallbacks and measurable quality. Never imply a recommendation guarantees savings or suitability. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 21 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 21 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 22 — Refund & Return Intelligence Agent

Primary owner: AI Lead + Backend Lead

Purpose

Assist refund and return review while leaving financial execution to authorized deterministic workflows.

Scope and work packages

• Retrieve related order, payment, return and policy records.

• Classify request reasons and identify missing evidence.

• Check merchant policy and jurisdiction-specific configuration where supplied.

• Recommend approve, reject or manual review with evidence.

• Require authorization, amount validation and idempotency before refund execution.

Expected deliverables

Refund case schema, agent, policy integration and sandbox workflow tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 22 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 22 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 22: Refund & Return Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Assist refund and return review while leaving financial execution to authorized deterministic workflows. Implement the following scope: Retrieve related order, payment, return and policy records. Classify request reasons and identify missing evidence. Check merchant policy and jurisdiction-specific configuration where supplied. Recommend approve, reject or manual review with evidence. Require authorization, amount validation and idempotency before refund execution. Deliverables: Refund case schema, agent, policy integration and sandbox workflow tests. Implementation directive: Build a refund assistant that prepares decisions and evidence; actual refunds must pass explicit policy and authorization checks. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 22 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 22 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 23 — Dispute Investigation Agent

Primary owner: AI Lead

Purpose

Assemble a coherent evidence package for payment dispute review.

Scope and work packages

• Link payment, order, invoice, fulfillment and communication records.

• Create a time-ordered event timeline with source references.

• Identify missing, contradictory or stale evidence.

• Draft a response for human review and prohibit fabricated evidence.

• Track submission deadlines and case status from authoritative sources.

Expected deliverables

Dispute case model, evidence graph, timeline generator and draft-response workflow.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 23 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 23 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 23: Dispute Investigation Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Assemble a coherent evidence package for payment dispute review. Implement the following scope: Link payment, order, invoice, fulfillment and communication records. Create a time-ordered event timeline with source references. Identify missing, contradictory or stale evidence. Draft a response for human review and prohibit fabricated evidence. Track submission deadlines and case status from authoritative sources. Deliverables: Dispute case model, evidence graph, timeline generator and draft-response workflow. Implementation directive: Build an evidence-first dispute assistant. Every statement must be traceable to a source record. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 23 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 23 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 24 — Business Operations Intelligence Agent

Primary owner: AI Lead

Purpose

Detect operational bottlenecks that affect payment and commerce outcomes.

Scope and work packages

• Analyze order processing, inventory signals, supplier events and support queues.

• Identify delays, recurring failures and downstream impact.

• Recommend operational actions with estimated impact and confidence.

• Avoid taking external actions without permissions and approval.

• Track whether recommendations improved the measured outcome.

Expected deliverables

Operations event model, agent, bottleneck reports and recommendation evaluation.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 24 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 24 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 24: Business Operations Intelligence Agent in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Detect operational bottlenecks that affect payment and commerce outcomes. Implement the following scope: Analyze order processing, inventory signals, supplier events and support queues. Identify delays, recurring failures and downstream impact. Recommend operational actions with estimated impact and confidence. Avoid taking external actions without permissions and approval. Track whether recommendations improved the measured outcome. Deliverables: Operations event model, agent, bottleneck reports and recommendation evaluation. Implementation directive: Build operations intelligence that connects business events to actionable, evidence-backed recommendations. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 24 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 24 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 25 — Historical Intelligence & Feature Store

Primary owner: AI Lead + Backend Lead

Purpose

Create governed historical data for analytics and model evaluation.

Scope and work packages

• Define approved feature datasets and data lineage.

• Separate training, validation and test sets by time and entity to reduce leakage.

• Track labels, label sources, confidence and correction history.

• Apply retention, deletion, access and anonymization policies.

• Version feature definitions and dataset snapshots.

Expected deliverables

Feature catalogue, lineage metadata, dataset versioning and governance tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 25 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 25 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 25: Historical Intelligence & Feature Store in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Create governed historical data for analytics and model evaluation. Implement the following scope: Define approved feature datasets and data lineage. Separate training, validation and test sets by time and entity to reduce leakage. Track labels, label sources, confidence and correction history. Apply retention, deletion, access and anonymization policies. Version feature definitions and dataset snapshots. Deliverables: Feature catalogue, lineage metadata, dataset versioning and governance tests. Implementation directive: Build a governed feature store. Do not treat every historical decision as a correct label. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 25 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 25 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 26 — Feedback Collection & Review Loop

Primary owner: AI Lead + UI Lead

Purpose

Capture human review outcomes and verified payment/commerce outcomes.

Scope and work packages

• Create review reasons and structured correction labels.

• Record who reviewed, when, evidence considered and final outcome.

• Protect feedback from malicious submissions and low-quality labels.

• Support dispute/appeal corrections and label confidence.

• Create an approval workflow before feedback enters training datasets.

Expected deliverables

Feedback API, reviewer UI, label provenance and data-quality checks.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 26 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 26 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 26: Feedback Collection & Review Loop in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + UI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Capture human review outcomes and verified payment/commerce outcomes. Implement the following scope: Create review reasons and structured correction labels. Record who reviewed, when, evidence considered and final outcome. Protect feedback from malicious submissions and low-quality labels. Support dispute/appeal corrections and label confidence. Create an approval workflow before feedback enters training datasets. Deliverables: Feedback API, reviewer UI, label provenance and data-quality checks. Implementation directive: Build a verified feedback loop. User feedback is a signal, not an automatic truth or immediate model update. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 26 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 26 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + UI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 27 — Adaptive Risk Model Evaluation

Primary owner: AI Lead

Purpose

Improve risk models through controlled evaluation and release management.

Scope and work packages

• Establish baseline metrics and threshold policies.

• Evaluate drift, calibration, subgroup performance and false-positive burden.

• Use shadow evaluation before model promotion.

• Version models, datasets, features and thresholds.

• Provide rollback and monitoring for model degradation.

Expected deliverables

Model evaluation pipeline, model registry metadata, promotion gates and rollback plan.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 27 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 27 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 27: Adaptive Risk Model Evaluation in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Improve risk models through controlled evaluation and release management. Implement the following scope: Establish baseline metrics and threshold policies. Evaluate drift, calibration, subgroup performance and false-positive burden. Use shadow evaluation before model promotion. Version models, datasets, features and thresholds. Provide rollback and monitoring for model degradation. Deliverables: Model evaluation pipeline, model registry metadata, promotion gates and rollback plan. Implementation directive: Implement controlled model improvement. No production model should self-deploy based only on its own generated feedback. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 27 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 27 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 28 — Predictive Intelligence Engine

Primary owner: AI Lead

Purpose

Add predictions for payment failures, churn, refund likelihood and operational delays.

Scope and work packages

• Define one prediction target at a time with clear label and horizon.

• Build leakage-safe training and validation datasets.

• Compare simple baselines with more complex models.

• Return prediction, uncertainty, explanation and model version.

• Monitor calibration, drift and business impact.

Expected deliverables

Prediction APIs, model cards, validation results and monitoring thresholds.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 28 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 28 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 28: Predictive Intelligence Engine in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Add predictions for payment failures, churn, refund likelihood and operational delays. Implement the following scope: Define one prediction target at a time with clear label and horizon. Build leakage-safe training and validation datasets. Compare simple baselines with more complex models. Return prediction, uncertainty, explanation and model version. Monitor calibration, drift and business impact. Deliverables: Prediction APIs, model cards, validation results and monitoring thresholds. Implementation directive: Build predictions only where data supports them. Show uncertainty and avoid presenting correlation as causation. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 28 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 28 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 29 — Recommendation Optimization & Experimentation

Primary owner: AI Lead + UI Lead

Purpose

Improve recommendation quality through measured experiments.

Scope and work packages

• Define experiment hypotheses and primary/guardrail metrics.

• Support randomized assignment where appropriate and avoid exposing sensitive segments.

• Measure relevance, diversity, conversion, complaints and downstream returns.

• Use safe fallback and stop conditions.

• Document experiment results and approved rollout decisions.

Expected deliverables

Experiment framework, recommendation metrics dashboard and rollout procedure.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 29 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 29 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 29: Recommendation Optimization & Experimentation in the Multi-Agent AI Payment Security Platform. Primary owner: AI Lead + UI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Improve recommendation quality through measured experiments. Implement the following scope: Define experiment hypotheses and primary/guardrail metrics. Support randomized assignment where appropriate and avoid exposing sensitive segments. Measure relevance, diversity, conversion, complaints and downstream returns. Use safe fallback and stop conditions. Document experiment results and approved rollout decisions. Deliverables: Experiment framework, recommendation metrics dashboard and rollout procedure. Implementation directive: Build a measurable recommendation optimization loop with guardrails and reproducible results. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 29 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 29 — Ownership, integration & handoff

Ownership

Primary owner: AI Lead + UI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 30 — Adversarial Testing & AI Security

Primary owner: Backend Lead + AI Lead

Purpose

Test the system against malicious prompts, data manipulation and agent misuse.

Scope and work packages

• Create threat models for prompt injection, tool misuse, spoofed messages and poisoned feedback.

• Test signed/authenticated messages, schema validation and capability enforcement.

• Sandbox untrusted content and restrict tool/network access.

• Add replay, rate-limit, dependency and secret-exposure tests.

• Create red-team scenarios and remediation tracking.

Expected deliverables

Threat model, adversarial test suite, security findings and remediation evidence.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 30 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 30 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 30: Adversarial Testing & AI Security in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead + AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Test the system against malicious prompts, data manipulation and agent misuse. Implement the following scope: Create threat models for prompt injection, tool misuse, spoofed messages and poisoned feedback. Test signed/authenticated messages, schema validation and capability enforcement. Sandbox untrusted content and restrict tool/network access. Add replay, rate-limit, dependency and secret-exposure tests. Create red-team scenarios and remediation tracking. Deliverables: Threat model, adversarial test suite, security findings and remediation evidence. Implementation directive: Perform defensive testing only on owned systems and test environments. Never create malware or unauthorized access tooling. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 30 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 30 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead + AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 31 — PayPal Developer & Sandbox Integration

Primary owner: Backend Lead

Purpose

Connect the platform to PayPal using supported current APIs and sandbox credentials.

Scope and work packages

• Create a PayPal developer app and separate sandbox buyer/seller accounts.

• Implement OAuth token acquisition, secure caching and expiry handling.

• Integrate the relevant Orders, Invoicing and other APIs actually needed by the MVP.

• Validate request/response schemas, currency, amounts and API error handling.

• Keep credentials server-side and document required scopes and account limitations.

Expected deliverables

PayPal adapter, sandbox test scripts, credential guide and integration evidence.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 31 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 31 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 31: PayPal Developer & Sandbox Integration in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Connect the platform to PayPal using supported current APIs and sandbox credentials. Implement the following scope: Create a PayPal developer app and separate sandbox buyer/seller accounts. Implement OAuth token acquisition, secure caching and expiry handling. Integrate the relevant Orders, Invoicing and other APIs actually needed by the MVP. Validate request/response schemas, currency, amounts and API error handling. Keep credentials server-side and document required scopes and account limitations. Deliverables: PayPal adapter, sandbox test scripts, credential guide and integration evidence. Implementation directive: Implement only documented current PayPal APIs. Verify every endpoint and SDK version against official documentation before coding. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 31 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 31 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 32 — Payment Risk Gateway & Orchestration

Primary owner: Backend Lead + AI Lead

Purpose

Connect risk assessment to a safe payment decision workflow.

Scope and work packages

• Accept payment intent/request and validate amount, currency, merchant and idempotency key.

• Collect only permitted risk features and call the orchestrator with deadlines.

• Run deterministic policy evaluation on validated agent outputs.

• Support allow, step-up, review or deny outcomes.

• Execute only approved PayPal actions and read back authoritative status.

Expected deliverables

Payment risk gateway, sequence diagram, policy tests and sandbox end-to-end flow.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 32 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 32 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 32: Payment Risk Gateway & Orchestration in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead + AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Connect risk assessment to a safe payment decision workflow. Implement the following scope: Accept payment intent/request and validate amount, currency, merchant and idempotency key. Collect only permitted risk features and call the orchestrator with deadlines. Run deterministic policy evaluation on validated agent outputs. Support allow, step-up, review or deny outcomes. Execute only approved PayPal actions and read back authoritative status. Deliverables: Payment risk gateway, sequence diagram, policy tests and sandbox end-to-end flow. Implementation directive: Build a safe payment gateway workflow. AI recommendations never directly authorize or claim payment completion. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 32 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 32 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead + AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 33 — PayPal Webhooks & Reconciliation

Primary owner: Backend Lead

Purpose

Receive payment events securely and reconcile local state with PayPal.

Scope and work packages

• Verify webhook authenticity using PayPal's documented verification mechanism.

• Validate event type, resource identity and event timestamp.

• Handle duplicate and out-of-order events idempotently.

• Use authenticated API readback where needed to confirm current state.

• Reconcile local payment ledger and expose discrepancies for investigation.

Expected deliverables

Webhook handler, reconciliation job, event tests and discrepancy dashboard.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 33 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 33 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 33: PayPal Webhooks & Reconciliation in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Receive payment events securely and reconcile local state with PayPal. Implement the following scope: Verify webhook authenticity using PayPal's documented verification mechanism. Validate event type, resource identity and event timestamp. Handle duplicate and out-of-order events idempotently. Use authenticated API readback where needed to confirm current state. Reconcile local payment ledger and expose discrepancies for investigation. Deliverables: Webhook handler, reconciliation job, event tests and discrepancy dashboard. Implementation directive: Implement secure webhook processing. Treat incoming events according to documented trust and verification rules. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 33 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 33 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 34 — Financial Execution Safety & Authorization

Primary owner: Backend Lead

Purpose

Protect money movement with strict validation and authorization boundaries.

Scope and work packages

• Use integer minor units or decimal-safe money types; never binary floating point for money calculations.

• Validate totals, currency, payee and operation state.

• Enforce idempotency, replay protection and state-machine transitions.

• Separate create, authorize, capture, void and refund permissions.

• Require approval and dual control for configured high-impact actions.

Expected deliverables

Financial state machine, authorization matrix, idempotency tests and audit evidence.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 34 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 34 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 34: Financial Execution Safety & Authorization in the Multi-Agent AI Payment Security Platform. Primary owner: Backend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Protect money movement with strict validation and authorization boundaries. Implement the following scope: Use integer minor units or decimal-safe money types; never binary floating point for money calculations. Validate totals, currency, payee and operation state. Enforce idempotency, replay protection and state-machine transitions. Separate create, authorize, capture, void and refund permissions. Require approval and dual control for configured high-impact actions. Deliverables: Financial state machine, authorization matrix, idempotency tests and audit evidence. Implementation directive: Implement deterministic financial controls and test duplicate requests, races, retries and partial failures. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 34 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 34 — Ownership, integration & handoff

Ownership

Primary owner: Backend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 35 — Main Command Center UI

Primary owner: Frontend Lead

Purpose

Build the main user-facing operational dashboard.

Scope and work packages

• Create navigation, organization context and role-aware dashboard.

• Show payment status, risk queues, agent health and operational alerts.

• Use accessible charts, tables, filters and responsive layouts.

• Provide loading, empty, error and stale-data states.

• Connect to typed API contracts and avoid placing secrets in the browser.

Expected deliverables

React dashboard, design system, API integration and accessibility checks.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 35 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 35 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 35: Main Command Center UI in the Multi-Agent AI Payment Security Platform. Primary owner: Frontend Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Build the main user-facing operational dashboard. Implement the following scope: Create navigation, organization context and role-aware dashboard. Show payment status, risk queues, agent health and operational alerts. Use accessible charts, tables, filters and responsive layouts. Provide loading, empty, error and stale-data states. Connect to typed API contracts and avoid placing secrets in the browser. Deliverables: React dashboard, design system, API integration and accessibility checks. Implementation directive: Build a polished, responsive command center with clear data provenance and no fake live metrics. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 35 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 35 — Ownership, integration & handoff

Ownership

Primary owner: Frontend Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 36 — Agent Intelligence Workspace UI

Primary owner: Frontend Lead + AI Lead

Purpose

Give operators visibility into agent identities, work and evidence.

Scope and work packages

• Build agent directory, profile, health and permission views.

• Show execution traces, message flow, model/version and tool calls.

• Display evidence, confidence, uncertainty and disagreement between agents.

• Allow authorized operators to pause/revoke agents and adjust approved configuration.

• Never expose secrets, hidden chain-of-thought or unrestricted tool controls.

Expected deliverables

Agent workspace, execution timeline, permission screens and role-based tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 36 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 36 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 36: Agent Intelligence Workspace UI in the Multi-Agent AI Payment Security Platform. Primary owner: Frontend Lead + AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Give operators visibility into agent identities, work and evidence. Implement the following scope: Build agent directory, profile, health and permission views. Show execution traces, message flow, model/version and tool calls. Display evidence, confidence, uncertainty and disagreement between agents. Allow authorized operators to pause/revoke agents and adjust approved configuration. Never expose secrets, hidden chain-of-thought or unrestricted tool controls. Deliverables: Agent workspace, execution timeline, permission screens and role-based tests. Implementation directive: Build an explainability workspace showing observable evidence and decisions, not private internal reasoning traces. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 36 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 36 — Ownership, integration & handoff

Ownership

Primary owner: Frontend Lead + AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 37 — Fraud Investigation & Analytics Workspace

Primary owner: Frontend Lead + AI Lead

Purpose

Provide a complete analyst workflow for investigating alerts and cases.

Scope and work packages

• Build case queue, filters, transaction detail and evidence timeline.

• Show agent findings, policy result, model version and reviewer actions.

• Support assign, comment, request evidence, approve/reject review and escalation.

• Add trend analytics for false positives, confirmed outcomes and review time.

• Ensure every reviewer action is permission-checked and audited.

Expected deliverables

Investigation UI, case workflow, analytics dashboard and end-to-end tests.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 37 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 37 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 37: Fraud Investigation & Analytics Workspace in the Multi-Agent AI Payment Security Platform. Primary owner: Frontend Lead + AI Lead. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Provide a complete analyst workflow for investigating alerts and cases. Implement the following scope: Build case queue, filters, transaction detail and evidence timeline. Show agent findings, policy result, model version and reviewer actions. Support assign, comment, request evidence, approve/reject review and escalation. Add trend analytics for false positives, confirmed outcomes and review time. Ensure every reviewer action is permission-checked and audited. Deliverables: Investigation UI, case workflow, analytics dashboard and end-to-end tests. Implementation directive: Build an evidence-led analyst experience. Make it easy to understand why a case was flagged and what remains uncertain. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 37 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 37 — Ownership, integration & handoff

Ownership

Primary owner: Frontend Lead + AI Lead. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 38 — End-to-End Integration & Contract Validation

Primary owner: All members; Team Lead coordinates

Purpose

Integrate modules through stable contracts and prove complete user journeys.

Scope and work packages

• Freeze API/event schemas for the integration window.

• Run contract tests between frontend, backend, agent services and PayPal adapter.

• Test payment assessment, review, execution, webhook update and reconciliation.

• Test agent timeout, partial result, contradictory result and broker outage.

• Track defects by severity and block release on critical security/payment failures.

Expected deliverables

Integrated release candidate, contract test report, end-to-end demo scripts and defect register.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 38 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 38 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 38: End-to-End Integration & Contract Validation in the Multi-Agent AI Payment Security Platform. Primary owner: All members; Team Lead coordinates. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Integrate modules through stable contracts and prove complete user journeys. Implement the following scope: Freeze API/event schemas for the integration window. Run contract tests between frontend, backend, agent services and PayPal adapter. Test payment assessment, review, execution, webhook update and reconciliation. Test agent timeout, partial result, contradictory result and broker outage. Track defects by severity and block release on critical security/payment failures. Deliverables: Integrated release candidate, contract test report, end-to-end demo scripts and defect register. Implementation directive: Integrate all modules without bypassing contracts. Demonstrate full workflows and failure recovery in sandbox. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 38 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 38 — Ownership, integration & handoff

Ownership

Primary owner: All members; Team Lead coordinates. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 39 — Security, Performance & Reliability Validation

Primary owner: All members; Backend Lead coordinates

Purpose

Validate security, correctness, latency, capacity and recovery.

Scope and work packages

• Run dependency and static analysis scans, secrets scanning and authorization tests.

• Run load, concurrency, soak and fault-injection tests.

• Measure agent latency, payment API latency, queue lag and end-to-end decision time.

• Test backup/restore, database migrations, retry storms and webhook replay.

• Review privacy, retention, audit completeness and model fairness/quality.

Expected deliverables

Security test report, performance baseline, recovery evidence and remediation sign-off.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 39 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 39 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 39: Security, Performance & Reliability Validation in the Multi-Agent AI Payment Security Platform. Primary owner: All members; Backend Lead coordinates. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Validate security, correctness, latency, capacity and recovery. Implement the following scope: Run dependency and static analysis scans, secrets scanning and authorization tests. Run load, concurrency, soak and fault-injection tests. Measure agent latency, payment API latency, queue lag and end-to-end decision time. Test backup/restore, database migrations, retry storms and webhook replay. Review privacy, retention, audit completeness and model fairness/quality. Deliverables: Security test report, performance baseline, recovery evidence and remediation sign-off. Implementation directive: Run authorized defensive tests on the project environment. Document limitations and fix critical/high findings before release. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 39 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 39 — Ownership, integration & handoff

Ownership

Primary owner: All members; Backend Lead coordinates. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

PHASE 40 — Deployment, Documentation & Hackathon Demonstration

Primary owner: Team Lead + all members

Purpose

Package a reproducible, judge-friendly demo and final project submission.

Scope and work packages

• Deploy a sandbox-only environment with seeded synthetic data and clear demo reset.

• Provide setup instructions, architecture diagrams, API documentation and threat model.

• Document AI models, model use, data sources, limitations and human approval controls.

• Create a concise demo path showing multi-agent collaboration and verified PayPal sandbox state.

• Prepare license, acknowledgements, team roles, video and submission checklist.

Expected deliverables

Release candidate, hosted demo, complete documentation, demo video and submission package.

Definition of done

• The module has a documented owner, inputs, outputs, dependencies and failure behavior.

• The implementation is merged through a reviewed pull request and passes required automated checks.

• A reviewer can reproduce the result using the phase documentation and test instructions.

• No unresolved critical security, privacy or payment correctness issue remains.

PHASE 40 — Implementation specification

A. Interfaces and contracts

• Define typed request, response and event schemas before implementation.

• Specify required fields, optional fields, validation rules, maximum sizes and versioning.

• Define authorization requirements and data classification for every input.

• Document timeout, retry, idempotency and error semantics.

• Provide example valid, invalid, partial and stale payloads.

B. Rust/backend implementation expectations

• Use Rust types to make invalid states difficult to represent.

• Return explicit error types; avoid panics for expected runtime failures.

• Apply bounded concurrency, timeouts, cancellation and resource limits.

• Use structured tracing with correlation IDs and redact sensitive fields.

• Add unit tests for business rules and integration tests for persistence or external boundaries.

C. Frontend expectations (where applicable)

• Use typed API clients and shared contract definitions.

• Provide loading, empty, error, stale and success states.

• Enforce role-aware presentation, but never rely on frontend checks for authorization.

• Make critical decisions understandable with evidence, reason codes and clear next steps.

• Ensure keyboard navigation, responsive layout and accessible labels.

D. AI/model expectations (where applicable)

• Treat model output as untrusted and validate it against a strict schema.

• Record model identifier/version, prompt/template version and evidence references.

• Set bounded token, latency and tool-call budgets.

• Provide fallback behavior when the model is unavailable or output is invalid.

• Do not train or update a production model automatically from unverified feedback.

PHASE 40 — Copy-ready AI coding prompt

Paste this prompt into the coding agent assigned to this phase. The coding agent must inspect the current repository first and preserve existing work.

```
You are a senior engineer implementing PHASE 40: Deployment, Documentation & Hackathon Demonstration in the Multi-Agent AI Payment Security Platform. Primary owner: Team Lead + all members. Project context: Rust backend (Axum/Tokio), React + TypeScript frontend, PostgreSQL, NATS for events where justified, and PayPal Sandbox for payment integration. Objective: Package a reproducible, judge-friendly demo and final project submission. Implement the following scope: Deploy a sandbox-only environment with seeded synthetic data and clear demo reset. Provide setup instructions, architecture diagrams, API documentation and threat model. Document AI models, model use, data sources, limitations and human approval controls. Create a concise demo path showing multi-agent collaboration and verified PayPal sandbox state. Prepare license, acknowledgements, team roles, video and submission checklist. Deliverables: Release candidate, hosted demo, complete documentation, demo video and submission package. Implementation directive: Prepare a reliable, transparent demonstration. Never imply real-money processing or fraud prevention performance that has not been verified. Before coding, inspect the existing repository and report the current architecture, relevant files, contracts and dependencies. Do not overwrite unrelated work. Propose a file-by-file implementation plan and identify any contract changes that require approval from the Team Lead. Implement production-minded code with typed interfaces, secure defaults, validation, structured errors, timeouts, bounded concurrency and traceable logs. Keep secrets server-side and use synthetic or sandbox data. For money, use decimal-safe amounts and explicit currencies. For payment operations, use idempotency, authoritative status readback and explicit authorization. AI agents may recommend but must not bypass the deterministic Rust policy engine. Add unit, integration and negative tests, including malformed input, missing data, duplicate requests, timeouts, partial failure and unauthorized access as relevant. Add documentation, configuration examples, migration or schema changes if needed, and a runnable demo/test command. Do not claim completion unless tests actually pass. At the end, provide: files changed, design decisions, API/event contracts, tests run and results, security considerations, known limitations, manual verification steps, and a pull-request summary. If a requirement is ambiguous or an external API capability is unverified, stop and ask for clarification rather than inventing behavior.
```

Required output from the coding agent

- Repository inspection summary and implementation plan.
- File-by-file changes with explanation of ownership and boundaries.
- Typed API/event contracts and configuration changes.
- Tests added, commands executed and actual pass/fail results.
- Security and privacy considerations, including rejected unsafe shortcuts.
- Known limitations, manual test steps and pull-request description.

Team instruction

The assigned owner implements the module. The Team Lead approves scope and interface changes. The relevant partner reviews compatibility. Do not merge until tests and documentation are included.

PHASE 40 — Testing, security & acceptance

Functional acceptance

• Happy path: representative valid input produces the expected typed output.

• Invalid input: missing, malformed, oversized or unsupported values are rejected safely.

• Boundary cases: empty collections, maximum supported values, stale timestamps and unknown versions are handled.

• Partial failure: unavailable dependency, timeout or interrupted operation produces a recoverable state.

• Repeatability: repeated execution does not create duplicate side effects.

Security acceptance

• Authentication and authorization are enforced at the trusted backend boundary.

• Inputs from users, agents, events and external APIs are treated as untrusted.

• Secrets, tokens and sensitive personal data are not written to logs or frontend bundles.

• Tenant boundaries and least-privilege capability scopes are tested.

• Security-relevant decisions and actions have an auditable trace.

Quality and performance acceptance

• Measure latency and resource use against a documented local baseline.

• Use deterministic fixtures for repeatable tests.

• Record model quality metrics when ML is involved; do not use accuracy alone.

• Test accessibility and responsive behavior for UI phases.

• Document known limitations and unresolved risks rather than hiding them.

Evidence checklist

• Test report or CI run link.

• Screenshots or recorded demo for user-visible functionality.

• API/event examples and schema version.

• Security review notes and remediation status.

• Owner and reviewer sign-off.

PHASE 40 — Ownership, integration & handoff

Ownership

Primary owner: Team Lead + all members. The Team Lead (Sanjay, provisional) coordinates dependencies and accepts the phase handoff. Member names should be replaced with the actual names before work starts.

Integration dependencies

• Upstream dependencies must be identified and pinned to an agreed contract/version.

• Downstream consumers must receive sample payloads and error semantics.

• Schema changes require a versioning and migration plan.

• Feature flags should protect incomplete workflows from appearing as production-ready.

• The integration branch must remain buildable after the phase is merged.

Handoff package

• Module README with purpose, setup, configuration and test commands.

• API/event schema and example payloads.

• Known limitations, threat assumptions and operational alerts.

• Database migration/rollback notes, if applicable.

• Demo script and evidence of acceptance tests.

Checkpoint sign-off

Role	Name	Sign-off / date
Phase owner	_____	_____
Technical reviewer	_____	_____
Team Lead	Sanjay (provisional)	_____

Phase status: ■ Not started ■ In progress ■ In review ■ Accepted ■ Blocked

Blockers / decisions / follow-up: _____

Master phase tracking register

Use this table during weekly planning. Replace the provisional team labels with actual names and add GitHub issue/PR links.

Phase	Title	Owner	Status	PR / Evidence
1	Product Requirements & Problem Definition	Team / Shared	Not started	
2	System Architecture & Technical Design	Team / Shared	Not started	
3	Repository, Workspace & Developer Experience	Backend Lead	Not started	
4	Database Architecture & Data Governance	Backend Lead	Not started	
5	Authentication, Authorization & Tenant Isolation	Backend Lead	Not started	
6	Rust Backend Core & API Foundation	Backend Lead	Not started	
7	Agent Communication & Event Bus	AI Lead	Not started	
8	Agent Identity & Capability Registry	AI Lead	Not started	
9	Deterministic Policy & Decision Engine	Backend Lead	Not started	
10	Audit, Observability & Incident Telemetry	Backend Lead	Not started	
11	Master Agent Orchestrator	AI Lead	Not started	
12	Transaction Pattern Analyst Agent	AI Lead	Not started	
13	Identity Verification Intelligence Agent	AI Lead	Not started	
14	Merchant Risk Intelligence Agent	AI Lead	Not started	
15	Device & Session Intelligence Agent	AI Lead	Not started	
16	Behavior Sequence Monitor Agent	AI Lead	Not started	
17	Collaborative Risk Aggregation Engine	AI Lead	Not started	
18	Payment Intelligence Agent	AI Lead	Not started	
19	Checkout Optimization Agent	AI Lead	Not started	
20	Customer Intelligence Agent	AI Lead	Not started	
21	Recommendation Engine	AI Lead	Not started	
22	Refund & Return Intelligence Agent	AI Lead	Not started	
23	Dispute Investigation Agent	AI Lead	Not started	
24	Business Operations Intelligence Agent	AI Lead	Not started	
25	Historical Intelligence & Feature Store	AI Lead	Not started	
26	Feedback Collection & Review Loop	AI Lead	Not started	
27	Adaptive Risk Model Evaluation	AI Lead	Not started	
28	Predictive Intelligence Engine	AI Lead	Not started	
29	Recommendation Optimization & Experimentation	AI Lead	Not started	
30	Adversarial Testing & AI Security	AI Lead	Not started	
31	PayPal Developer & Sandbox Integration	Backend Lead	Not started	
32	Payment Risk Gateway & Orchestration	AI Lead	Not started	

Phase	Title	Owner	Status	PR / Evidence
33	PayPal Webhooks & Reconciliation	Backend Lead	Not started	
34	Financial Execution Safety & Authorization	Backend Lead	Not started	
35	Main Command Center UI	Frontend Lead	Not started	
36	Agent Intelligence Workspace UI	AI Lead	Not started	
37	Fraud Investigation & Analytics Workspace	AI Lead	Not started	
38	End-to-End Integration & Contract Validation	Team / Shared	Not started	
39	Security, Performance & Reliability Validation	Backend Lead	Not started	
40	Deployment, Documentation & Hackathon Demonstration	Team / Shared	Not started	

Final release checklist

- All 40 phase owners and reviewers are named.
- Architecture and API contracts are versioned and approved.
- No unresolved critical/high security or payment correctness defects remain.
- PayPal Sandbox credentials are stored only in server-side secret storage.
- Webhook signature verification and duplicate-event handling are tested.
- Payment state is confirmed from authoritative PayPal responses/readbacks.
- Agent outputs are schema-validated and policy-gated.
- Model versions, datasets and evaluation reports are documented.
- Synthetic demo data is clearly identified as synthetic.
- The hosted demo has a reset path and does not expose real customer information.
- README, license, architecture, setup, API and security documents are complete.
- Demo video accurately explains the problem, agent collaboration, PayPal integration and limitations.

Suggested first milestone

Deliver a working sandbox prototype with the master orchestrator, transaction analyst, identity signal analyst, merchant intelligence, device/session intelligence, behavior monitor, deterministic policy engine, PayPal Sandbox order flow and a basic React investigation dashboard. Only expand to the remaining commerce and learning modules after this end-to-end path is reliable.

End of masterbook